Practical multiverse debugging through user-defined reductions

Matthias Pasquier, Ciprian Teodorov, Frédéric Jouault, Matthias Brun, Luka Le Roux, Loïc Lagadec

Practical Multiverse Debugging through User-defined Reductions

Application to UML Models

Matthias Pasquier
ERTOSGENER
Angers, France
matthias.pasquier@ertosgener.com

Ciprian Teodorov
Lab-STICC CNRS UMR 6285
ENSTA Bretagne
Brest, France
ciprian.teodorov@ensta-bretagne.fr

Frédéric Jouault
ESEO
Angers, France
frederic.jouault@eseo.fr

Matthias Brun
ESEO
Angers, France
matthias.brun@eseo.fr

Luka Le Roux
Lab-STICC CNRS UMR 6285
ENSTA Bretagne
Brest, France
luka.le_roux@ensta-bretagne.fr

Loïc Lagadec
Lab-STICC CNRS UMR 6285
ENSTA Bretagne
Brest, France
loic.lagadec@ensta-bretagne.fr

ABSTRACT

Multiverse debugging is an extension of classical debugging methods, particularly adapted to non-deterministic systems. Recently, a language-independent formalization was proposed. Moreover, multiverse debugging is particularly beneficial for specification and design languages, such as UML. However, this method suffers from scalability issues during breakpoint lookup. This problem arises due to the exhaustive exploration performed on the potentially infinite state-space of the system.

In this paper, we tackle this problem by introducing Reduced Multiverse Debugging, an extension proposing a way for the user to define reduction policies used during breakpoint lookup. We enrich the formalization of multiverse debugging with a modular breakpoint lookup strategy, which allows the integration of the reduction policy. We validate our approach by implementing a practical UML Statechart debugger in the AnimUML web framework. We show several ways the reduction can be applied, using methods such as predicate abstraction for breakpoint lookup on an infinite state-space, removing irrelevant variables, or creating classes of equivalent values. Moreover, we show the possibility to integrate probabilistic reduction strategies. Relying on hash collisions, these strategies can be iteratively refined to increase precision.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**.

KEYWORDS

multiverse debugging, model analysis, concurrency, abstraction

1 INTRODUCTION

Debugging is an important part of the development life cycle. By allowing the developer to explore programs interactively, it can be useful to find faults as well as simply get a better comprehension of its inner workings. As the complexity of programs increased, this process has also evolved to adapt to these new difficulties.

Omniscient debugging [22] allows us to return back on the execution trace to previously visited configurations. Multiverse debugging [23] has been introduced to enable the exploration of concurrent actor-based formalisms. This technique becomes important for the debug of multi-threaded actor systems, where the different execution schedules can hide bugs. For high-level specification languages, such as UML, this feature is necessary due to the intrinsic non-determinism, which besides scheduling allows capturing entire families of implementations.

The formalization of a unified debug semantics targeting non-deterministic specification languages, such as UML [26], was introduced by Besnard et al. [6, Section 5.3, pp. 82–87]. This formalization generalizes multiverse debugging rendering it independent of the language. However it does not define the breakpoint finder strategy, and faces the problem of system complexity when applied to real world examples. This is due to the exhaustive exploration of the potentially infinite space-state needed during breakpoint lookup.

In this paper, we address this problem by introducing Reduced Multiverse Debugging (RMD), an extension proposing a way for the user to define reduction policies used during breakpoint lookup. These reduction policies introduce under-approximations during breakpoint search to significantly improve the scalability of multiverse debugging, thus rendering the approach usable in practice. This allows to use methods like predicate abstraction [11, 15], removing irrelevant variables [2], creating classes of equivalent values or use probabilistic reduction methods, such as bitstate hashing

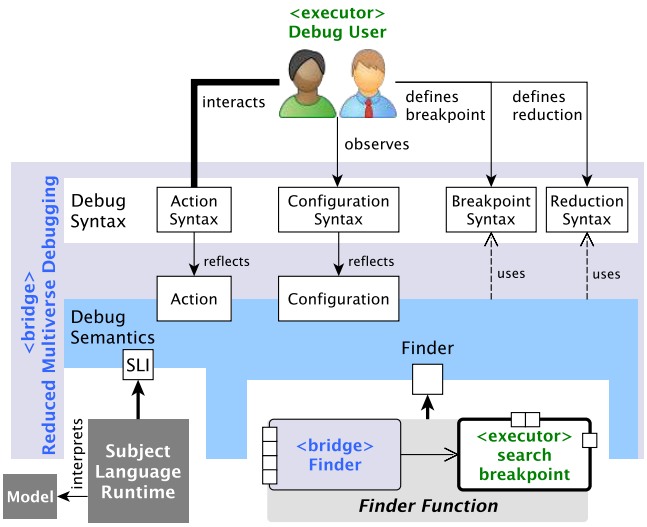

Figure 1: Architecture overview

[16] and hashcompaction [33]. Moreover, we present a complete formalization that also cover the breakpoint lookup strategy.

Figure 1 overviews the architecture of a RMD tool, including the actions available to the end user. As debugging remains a human activity, the user is represented as the "debug user" executor. The interactions during debugging are classified as: the actions (the orders), observing the configuration (state) of the system, defining breakpoints and defining reduction policies. The user actions include stepping through the system's behavior, jumping back to a previously observed configuration, selecting between a non-deterministic choice and running to a breakpoint from the current configuration. The breakpoint definition syntax allows the user to define the stop criterion. Lastly, the user can define a reduction policy that is used during breakpoint lookup.

The debugger semantics, can be seen as a bridge between the user and the debugged model. The debugger is dependent on the subject language runtime which captures the dynamic semantics of the model. In our case, the subject semantics is encapsulated in a Semantic Language Interface (SLI), that provides language-agnostic observation and control services. Besides the subject language, the Figure 1 emphasizes the modularity of our approach and the importance of the breakpoint *Finder Function* by representing it as an independent unit. When debugging a deterministic model, this function unravels the single execution path possible. However, multiverse debugging forces this function to search through multiple non-deterministic branches, a typical functionality of model-checking, which brings the state-space explosion problem. The run to breakpoint action of the RMD is realized by calling the Finder Function passing the subject language runtime as parameter along with the breakpoint and the reduction policy.

We validate the practicality of the approach by implementing a UML debugger, integrated in the AnimUML modeling environment. To evaluate our contribution we discuss two debugging scenarios. The first scenario, based on the Lamport's bakery algorithm [20], emphasizes the importance of using reduction strategies for the analysis of infinite state-space models. The second scenario, more performance oriented, illustrates both the scalability and the flexibility of the approach by showing that the precision of the analysis can be tuned by varying the reduction strategy.

In summary, this paper's contributions with respect to [6] are the following: 1) the finder function becomes modular, 2) it can now leverage a reduction policy, which has never been done in the context of multiverse debugging to the best of the authors' knowledge, and 3) all of this has been validated by extending AnimUML, and showing how it works on typical scenarios.

Section 2 introduces related work. Section 3 overviews the formalization of the debugger semantics and the reduced breakpoint lookup strategy. In Section 4, we overview the UML implementation and discuss two illustrative examples. Section 5 evokes some limits of our approach. This paper concludes, in Section 6, providing some future research directions.

2 RELATED WORK

This work is related to the successive evolution of debugging techniques. Omniscient debugging [22] proposes a way to prevent unnecessary restarts of debugging sessions by allowing going backward on the execution trace, creating the possibility for the user to go "back in time". This technique has been adapted to be used with multiple general purpose languages, such as C [8] or Java [14]. There also exist debuggers for languages closer to our approach, such as [21], extending the fUML semantics to support debugging operations including back stepping, or [4], where the authors implement MDebugger[3], a debugger for UML-RT models into a live modeling environment. In [24], the authors implement omniscient debugging for statecharts. The K debugger, presented in [32], automatically derives a debugger from the language semantics. However, it requires the use of the kframework [31] for specifying the language semantics. In [9], the authors present a language independent omniscient debugger with an implementation based on the GEMOC Studio, which offers multiple possibilities for defining the subject language semantics. However, all of these approaches are limited to linear execution traces.

Multiverse debugging [23] extends this concept by integrating the non-deterministic nature of modern programs directly into the possible debugging actions. In addition to jumping back to previously explored configurations, it is now possible to select through multiple actions, creating a branching history. However, the actor-based subject language clearly restricted the scope to implementation languages. Furthermore, the authors discuss the possibility of using static analysis for improving scalability, but leave these aspects for future works. Our contribution act on this scalability problem through user-defined under-approximations.

In [6, 7], the authors propose a generic multiverse debugger formalization, that generalizes the approach to non-deterministic specification languages, such as UML. However, this formalization misses the importance of the breakpoint lookup strategy, including the problem of state-space explosion. In this paper, we complete this formalization, including a way to scale the approach during breakpoint search.

AnimUML [17] introduces a unified approach for the verification and monitoring of executable UML. To achieve this, the authors

rely on a transition system semantics similar to the one used by our approach. Moreover, the paper introduces the first UML-specific graphical user interface (GUI) for multiverse debugging. However, once again the lack of scalability is still not addressed. Our solution extends the AnimUML multiverse debugger GUI to allow the definition of the reduction strategies, by extending the watch expression syntax. Moreover, we improve the user feedback by adding new visualization exposing the branching execution history.

The model-checking [10] and symbolic execution [5] literature is rich in abstraction techniques to counteract the state-space explosion problem. Over-approximations analyze supersets of the state-space, but can introduce false positives (report errors not present in the original model). Under-approximations focus on subsets of the state-space, but can introduce false negatives (miss errors). An abstraction is precise with respect to a given property if it does not introduce neither false positives nor false negatives. Besides introducing false positives (which require further steps for confirmation), over-approximation techniques require either the modification of the model being debugged, or the replacement of the semantics (with an abstract one) or both. "On-the-fly" state-space reductions [27], are a class of under-approximation techniques, that can be implemented independently of the subject language semantics. Using an exploration method called "concrete search with abstract matching"[27] these abstractions can be lazily computed during the exploration of the concrete model. Our main contribution is to show that this approach brings a practical solution to the state-space explosion problem for multiverse debugging. Moreover, the reductions can be adapted to the needs of the user. Furthermore, this language-agnostic technique is fully compatible with the use of abstraction techniques requiring model transformations or support from the language runtime.

3 RMD FORMALIZATION

This section formally introduces a high-level generalization of the traditional debugging tools. This debugger specification abstracts away some implementation details, such as the structure of the trace, and the projection functions needed for the user interface.

All listings of this section will be presented in lean 3 [13], unless stated otherwise. Some of them may present a simplified syntax for better clarity, but the complete formalization can be found in the dedicated repository of this article[1].

3.1 The Semantic Language Interface

Before getting to the semantics of the debugger, we need to define the way it is interfaced with the program to be debugged. To this end we use the concept of Semantic Language Interface, as presented in Listing 1. Definition 3.1 also gives a description of the types used in this formalization. The SLI is composed of multiple elements: First, a structure called Semantic Transition Relation, abridged as STR, represents the interface to control the model execution. The *initial* function returns the set of all possible initial configurations. For a given configuration, the function *actions* returns the actions that can be executed next. Finally, the *execute* function executes an action from a given configuration, and returns the possible resulting configurations. The presence of sets in these functions allow us

[1]https://github.com/MatthiasPasquier97/reduced-multiverse-debugging

```
structure STR :=
    (initial: set C)
    (actions: C → set A)
    (execute: C → A → set C)
class Evaluate := (state: E → C → V)
class Reduce   := (state: R → C → α)
```

Listing 1: The semantic language interface definition

```
structure DebugConfig :=
    (current: option C)
    (history: set C)
    (options: set C)
```

Listing 2: The configuration of the G∀min∃ debugger

to use non-determinism in the language. Along the STR, the SLI includes two type classes[35] necessary for two additional functions. The *evaluate* function is used to evaluate an expression E over a configuration, returning the value of this expression. The function *reduce* allows to obtain a reduced configuration from a complete one, following the rules of the reduction syntax R.

Definition 3.1. The generic types used by the SLI

C the *configuration type* encodes the structured dynamic memory of the subject language;

A the *action type* symbolically reifies the link between two configurations in the subject language;

E the expression type captures the *configuration inspection syntax*[2] exposed by the SLI, used to define the breakpoints;

R the *reduction syntax*, exposed by the language interface to enable configuration reduction;

V the *value type* characterizes the configuration inspection results as values in the subject language semantic domains. Note that our formalization requires only boolean values;

α the *reduced configuration type* captures the configuration reduction results. Note that for self-representing languages[30] the α and V may overlap, however, as this is not the case in general we prefer to consider them separately.

3.2 Debugger as a Dependent Semantics

Without loss of generality, the G∀min∃ debugger, presented here, is parameterized over the subject language semantics, captured as a transition relation in the spirit of [7]. In the following, this will be referred as a Semantic Transition Relation (STR). Furthermore, by decoupling the debugger semantics from the subject language details, the G∀min∃ debugger is formally defined independently of the underlying language.

The G∀min∃ debugger is defined as a monitoring wrapper over the subject language semantics definition. The debugger itself exposes the STR interface. The debugger specification is based on two specific types: the configuration and the actions.

The debugger configuration, DebugConfig in Listing 2, is parameterized by the subject language configuration type (C) and simply captures the core state-variables of the debugger, namely

[2]The inspection syntax becomes the introspection syntax if it is reflectively exposed in the subject language syntax

```
inductive DebugAction
| step: A → DebugAction
| select: C → DebugAction
| jump: C → DebugAction
| run_to_breakpoint: DebugAction
```

Listing 3: Debugging actions, the abstract syntax of the debugger

the current state (*current*), the execution history (*history*) and the options of next configurations (*options*). The *current* configuration simply wraps the current configuration of the subject language in an option type. The *history* is the set of configurations encountered since the beginning of the debugging session. The *options* variable is necessary because of the non-determinism of the execute function in the subject language. It acts as a temporary configuration buffer from which the next configuration can be selected.

Please note that we relax the type of the *current* variable to an option type. This is not necessary in practice. However, for the purposes of this presentation it simplifies the specification by alleviating the need to "arbitrarily" choose one configuration from the options to set as "current" configuration. Furthermore, note that, for the purposes of this formalization, the *history* does not need more structure, and can simply be represented as a set. The type of *history* can be adapted to a specific type of debug: For exemple, omniscient debugging stores the history as a linear path while multiverse debugging uses a tree of potential executions.

The dynamic actions of the debugger are captured by the DebugAction type, illustrated in Listing 3, which introduces 4 core debug actions. The DebugAction is parameterized over the configurations (C) and actions (A) of the subject language. The *step* action syntactically describes the intention of executing a particular action of the wrapped STR. The *select* action captures the selection step necessary for resolving the non-deterministic execution of the wrapped STR. The *jump* action represents the intention of moving from the *current* configuration to an arbitrary configuration picked from the *history*. Finally, the *run_to_breakpoint* action represents the intention of running the underlying STR until a "breakpoint" is found.

To gain some intuition on the G∀min∃ debugger, Figure 2 shows an example of the debug actions allowed by our specification. The figure overlays the debug actions over the configuration 3 (the rounded rectangle with a thick border) of a simple transition system (the dashed blue lines indicate the transitions). The black dot denotes the initial state. The red dots denote the configurations that satisfy the breakpoint predicate. The $a1$ & $a2$ labels on the blue transitions explicitly indicate the original actions which link the configurations. The execution of the $a2$ action is non-deterministic. Moreover, the configurations 3 & 7 have action non-determinism (two actions are enabled). The gray rounded rectangles show the configurations unrolled by the debugger up to the current configuration. From the current configuration (configuration 3) five actions are enabled: *a)* two *step* actions, which execute the $a1$ and $a2$ actions of the original transition system; *b)* two *jump* actions, which enable jumping back to one of the previously discovered configurations; *c)* the *run_to_breakpoint* action, which discovers the trace ($8 \leftarrow 7 \leftarrow 5 \leftarrow 3$) of the original transition system. Conceptually, any subset

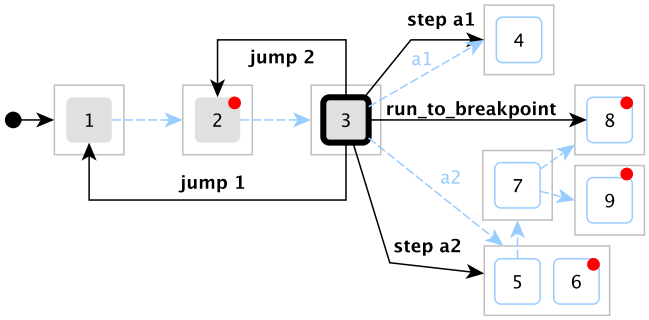

Figure 2: The G∀min∃ debug actions overlayed over a simple transition system. The number-labeled rounded rectangles represent the configurations of the subject-language. The dashed blue lines indicate the transitions (actions) in the subject-language. The boxes around the configurations indicate the potential non-determinism of the STR-execution function (returning a set of configurations). The configuration 3 (thick black border) is in the current slot of the debug configuration. The grayed configurations indicate that they are present in the history slot of the debug configuration.

```
def Finder
  [has_evaluate: Evaluate C E bool]
  [has_reduce:   Reduce C R α] :=
  STR C A → set C → E → R → list C
```

Listing 4: The signature of the breakpoint finder function

of all witness traces may be produced, as in [23]. For simplicity, our specification limits the scope of the run_to_breakpoint action to the first discovered witness. Thus, the configuration 6 and 9 are not seen by this action execution.

The Figure 3 shows an excerpt of the transition system induced by our debugger specification, defined in Listing 5, on the original transition system in Figure 2. The gray arrows separated by $\cdots$ indicate the missing actions. Each rounded rectangle represents one instance of *DebugConfig*, with its three slots. The effects of the five debug actions are emphasized with the text fuchsia. The *step* actions set the options slot to the configurations obtained by executing the original action. The *jump* actions, change the current slot, extend the trace and reset the options slot. The *run_to_breakpoint* action updates the current configuration and extends the trace with the witness trace. The *select* action changes the current slot, extends the trace and resets the options slot. In our example it is used to choose the configuration 6, between the two options (5 & 6) obtained by steping with $a2$.

For simplicity, in the following, we consider a single breakpoint of type E, which is obtained from the ambient environment. The breakpoint semantics is given over the subject-language configurations by the evaluate function. In general, the breakpoint language can be defined as a dependent language over E. For instance a propositional language with the atomic propositions typed by E. This can be useful if the subject language does not allow the evaluation of propositional logic expression over execution steps. Moreover,

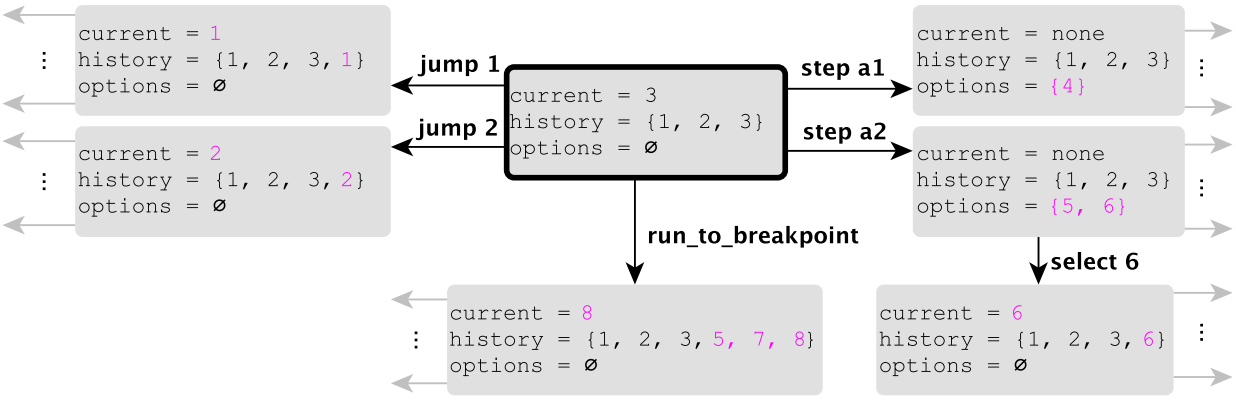

Figure 3: The debug transition system induced on the example in Figure 2

```
def ReducedMultiverseDebuggerBridge
    [has_evaluate: Evaluate C E bool]
    [has_reduce:   Reduce C R α]
(o: STR C A) (find: Finder C A E α) (break: E) (red: R)
: STR (DebugConfig C) (DebugAction C A) := {
   initial :=          rmdInitial o,
   actions := λ dc,    rmdActions o dc,
   execute := λ dc da, rmdExecute o find break red da dc  }
```

Listing 5: The STR specification of the RMD

in some situations it can be simpler to expose a primitive evaluation function, and let the clients (the debugger, in our case) handle higher-level expressions.

The strategy used for running to a breakpoint is abstracted away through the use of the *Finder* function, the signature is presented in Listing 4. This function requires the *Evaluate* and the *Reduce* functions[3] besides the subject-language STR[4]. Given those, the *Finder* performs a reduced multiverse exploration looking for a witness trace (list *C*) connecting one of the start configurations (set *C*), to one configuration satisfying the breakpoint predicate (E). The multiverse reduction policy is provided by the *R* argument. In section 3.4, we define this function modularly through the use of the SLI.

3.3 A Semantic Transition Relation for Debugging

Based on the previous definitions the semantics of the debugger can be captured as a STR, illustrated in Listing 5. The *ReducedMultiverseDebuggerBridge* is parameterized by: the subject STR (*o*) along with the related evaluation function, the finder function (*finder*), the breakpoint, and the abstraction function. The return type is a new STR that wraps the subject language semantics providing the debug actions. Please note that besides the arguments passed to the *rmdExecute*, this function (presented in Listing 8) also requires instances of the Evaluate (contrained to a bool result) and Reduce

[3]Please note that the *Evaluate* and *Reduce* functions are inferred by the lean typeclass resolutions mechanism.

[4]We choose to explicitly pass the subject language STR as an argument at the call site.

```
def rmdInitial {C A: Type} (o: STR C A): set (DebugConfig C)
:= {⟨none, ∅, o.initial⟩}
```

Listing 6: The initial state predicate of the G∀min∃ debugger

```
def rmdActions {C A: Type}
(o: STR C A): DebugConfig C → set (DebugAction C A)
| ⟨ current, history, options ⟩ :=
 let
   oa := { step   a | ∀ c,  current = some c → ∀ a ∈ (o.
     actions c) },
   sa := { select c | ∀ c ∈ options }
   ja := { jump   c | ∀ c ∈ history },
in oa ∪ sa ∪ ja ∪ { run_to_breakpoint }
```

Listing 7: The specification of the debugger "actions" function

typeclasses, which are automatically inferred by the lean typeclass resolution mechanism.

Listing 6 shows the definition of the *rmdInitial* function, which simply creates a singleton containing a *DebugConfig* with no current state, an empty history set, and the *options* variable set to the initial states obtained from the wrapped STR *o*.

The *rmdActions* function, shown in Listing 7, generates the set of enabled actions in a *DebugConfig dc*. In our case there are four sources of actions:

(1) the original action set *oa*, obtained from the subject STR *o* (only if the *current* slot is not empty). These actions are exposed through the *step* constructor.
(2) the select action set *sa*, induced by the elements in the *options* slot, which are encapsulated by the *select* constructor.
(3) the jump action set *ja*, induced by the *history* elements, which are encapsulated by the *jump* constructor.
(4) the run_to_breakpoint action.

The *rmdExecute* function, presented in Listing 8, defines the execute step semantics of our *unified_debugger*. There are five cases to consider. If the *current* slot is not empty (`dc.current = some c`) and the original action execution does not block (`o.execute c a ≠ ∅`), the execution of the step creates a new configuration with the

```
def rmdExecute {C A E R α: Type}
    [has_evaluate: Evaluate C E bool]
    [has_reduce:   Reduce C R α]
(o: STR C A) (find: Finder C A E R α) (break: E) (red: R)
: DebugAction C A → DebugConfig C → set (DebugConfig C)
| (step a)            ⟨ (some c), history, _ ⟩
    := { ⟨ none, history, o.execute c a ⟩ }
| (step a)            ⟨ (none), _, _ ⟩
    := ∅
| (select c)          ⟨ _, history, _⟩
    := { ⟨ c, { c } ∪ history, ∅ ⟩ }
| (jump c)            ⟨ _, history, _⟩
    := { ⟨ c, { c } ∪ history, ∅ ⟩ }
| (run_to_breakpoint) ⟨ (some c), history, _  ⟩
    := match (finder o { c } breakpoint reduction) with
    | []   := ∅
    | h::t := { ⟨ h, history ∪ { h } ∪ {x | x ∈ t}, ∅ ⟩ }
    end
| (run_to_breakpoint) ⟨     none, history, opts⟩
    := match (finder o opts breakpoint reduction) with
    | []   := ∅
    | h::t := { ⟨ h, history ∪ { h } ∪ {x | x ∈ t}, ∅ ⟩ }
    end
```

Listing 8: The specification of the "execute" function

options slot assigned to the results of calling the original execute function. Otherwise the execution of the step blocks. The current slot is cleared and the history is not modified. The *select* and *jump* executions are identical, the argument configuration is set in the *current* slot, the *history* is extended with the argument configuration and the *options* slot is emptied. The execution of a *run_to_breakpoint* is different based on the state of the *current* slot. If empty, then the *finder* function is invoked with the configurations from the *options* slot as starting point. If the *current* slot is not empty (`some c`), the finder starts the lookup for breakpoint from the configuration in this slot. If the finder locates a witness trace, linking the starting point with a configuration satisfying the breakpoint predicate, a new debug configuration is created with the witnessing configuration (the configuration that satisfies the breakpoint condition) in the *current* slot and a trace extended with the witness (`history ∪ { h } ∪ {x | x ∈ t}`). Otherwise the execution does not produce any configuration (it blocks).

Note that the execution function is deterministic, since it produces at most one new *DebugConfiguration* for each action. Furthermore, note that the blocking behavior of the *debugExecute* function is only a design choice. Nevertheless it prevents the introduction of "strange" debug configurations, like ⟨ `none, history, ∅` ⟩) which blocks all actions but *jump*.

3.4 A Modular Finder Function

The *Finder* function, shown as a gray box in Figure 4, is the workhorse of any debugger. It enables the user to jump forward in time, over the transition relation, to specific "points" of interest (identified by breakpoint predicates). In deterministic sequential languages, the *Finder* function corresponds to running the program until a breakpoint condition is satisfied. In general, however, the *Finder* needs

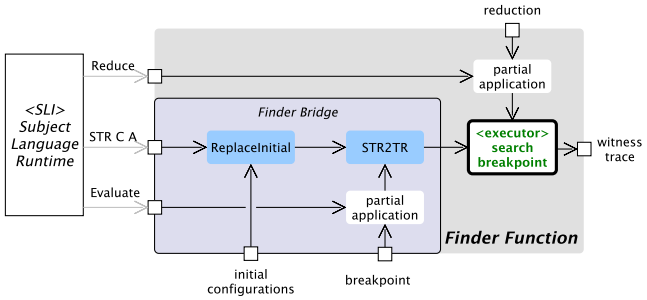

Figure 4: The architecture of the *Finder* function

to perform a reachability query on the transition system induced by the semantics. Relying on the functionalities exposed by the SLI, this functionality can be implemented in an ad-hoc manner by specializing any reachability algorithm to the peculiarities of the SLI interface. However, from our perspective, such an approach defeats the purpose of the SLI interface, which aims at decoupling the semantics from the algorithms to enable their independent evolution. From this point of view, we should not compromise neither on the semantics nor on the reachability algorithm, but rather exploit the SLI facilities to build a bridge between the two, like illustrated in Figure 4 (the blueish *Finder Bridge* rounded rectangle).

The first ingredient to consider is the subject language SLI (the left rectangle in Figure 4), from which we need the STR along with the evaluate and reduce functions. Inspecting the configurations through the evaluate predicate enables multiple usage scenarios by allowing a rich set of breakpoints, such as:

- Predicates related to the syntax tree (AST predicates), the run-to-line is one example, where the line number is a proxy for the AST node;
- Predicates on the configuration (state predicates), the run-to-assertion is an example;
- Mixed AST and state predicates, the conditional breakpoint is a example, which is a conjunction of a state-predicate and a AST predicate.

The Finder also uses multiple function to adapt this SLI to the search algorithm. The *STR2TR* allow to use the STR as a transition relation, noted *TR*, by combining the *actions* and *execute* functions into a single *next* function, that executes every possible action, and returns the union of all possible resulting configurations. The *partial application* uses the reduce and evaluate functions of the SLI to permanently take the given reduction and breakpoint and only accept a configuration as an input. Finally, the *replaceInitial* allow to change the initial configurations, as the run to breakpoint function may be launched from any arbitrary configuration during the debugging session.

We also need the executor *search_breakpoint*, which uses the function *search_breakpoint* described in Listing 9. This function performs a reachability query over a non-deterministic transition relation to find a configuration accepting the breakpoint expression, by calling the *dfs* function for each initial configurations.

The *dfs* function recursively explores the next possible configurations until one of them is an accepting configuration, ending the

```
1  def search_breakpoint(o: TR C) (reduce: C → α): list C :=
2      k = {}; wt = ();
3      for s ∈ o.initial do
4          if dfs o reduce s k wt then return wt
5      return wt
6  def dfs (o: TR C) (reduce: C → α)
7  (s: C) (k: set C) (wt: list C): bool :=
8      if o.accepting s then
9          wt = wt.append(s); return true
10     k = k ∪ { reduce s }
11     for t ∈ o.next s do
12         if (reduce t) ∉ k then
13             if dfs o red t k wt then
14                 wt = wt.append(s); return true
15     return false
```

Listing 9: Pseudo-code for the *search_breakpoint* function that performs a reduced reachability query over a non-deterministic transition relation with accepting states

search and returning the witness trace *wt*. If no further exploration is possible, the function returns false, indicating that no accepting state was found in this branch. If there is no accepting state obtained from any of the initial configurations, the breakpoint cannot be reached in the complete state space. To avoid verifying some identical branches multiple times, or keeping on exploring some loop in the state space, we compare each configuration obtained from the *next* function with a list of already discovered configurations, noted as *k* in this program. This is where our *reduce* function is used, as we do not use complete configurations directly, but we store and compare the reduced versions of these configurations.

The reduction function takes a configuration C as an input and can produce another type α as long as there is a possibility to check for equality between values of this type. It effectively performs state-space pruning during the search for breakpoints. The verification is exhaustive if: *a)* the reduce is injective; *b)* the reduce maps the configurations through symmetries to a quotient of the configuration space; *c)* the reduce function is proved to be a precise approximation of the configuration space. In general though, the reduce function performs a under-approximation of the configuration space. Thus, we can only reach real configurations of the system, making false positives impossible. A too eager reduction will lead to the impossibility to conclude on the reachability of the breakpoint, but cannot give false positives, allowing the user to relax the reduction if necessary.

Our goal in this article is to leverage the many possibilities offered by on-the-fly reductions in the context of debugging. Choosing and implementing the best reduction is not a trivial task and the best way to achieve it still needs further work. Nevertheless, we present some ways to choose reductions, depending on the model as well as the nature of the breakpoint we want to reach. Furthermore, the literature on the subject [2, 15, 16, 18, 33, 34] can also guide the user.

The first possibility is to use some probabilistic method. To this end, we use a hashing function for our reduction, creating a surjection between the big or potentially infinite number of configurations of the concrete systems and the smaller number of possible hashed values. We then obtain a finder function working on the principles of bitstate hashing [16] and stopping the exploration before it is complete. Deciding on the best hash size may require a bit of trial and error to be optimized, but the chosen values can be kept for the following debugging sessions.

It is also possible to use a targeted reduction to push the exploration towards parts that seem interesting. In the same way the debugging process allows us to choose manually the next steps to execute, which allows us to go faster to the parts of interest, we can do the same by defining an abstraction which ignores some elements of the concrete configuration as our reduction. This allows us to implement some classic strategies of model abstraction: It is possible to do dead variable reduction by ignoring a variable that has little to no interaction with the part of the model that interests us, like a logging variable. We can also find classes of equivalent variables, and only consider a change if the variable moves to another class.

Finally we can use reduction that are directly adapted to the situation of the specific model and language. Some examples would be to restrict the access to an uninteresting part of the model by selecting if some states are kept in the memory or not. If the language uses a mechanism of message passing, the event pool of each object can be problematic if it is unbounded, and we can help it by capping it, or limiting the number of similar events.

Note that multiple reduction functions can be combined through simple function composition, to further decompose the reduction strategy. For example, a symmetry reduction followed by a hashing function could be beneficial due to the smaller domain of the hashing function, which will translate in a lower collision rate.

For helping the user to implement these reductions, several elements are presented. Firstly, the evaluate function of the SLI allows to evaluate expressions on the system even if the configurations communicated through the STR are not transparent. Moreover, generic primitives can also be provided. For instance, we provide a hashing function for which users can choose hash bit length. This makes it trivial to use hash compaction in an iteratively refined manner. In the same way, reduction functions can evolve during the debugging process, just like breakpoint placement can be refined in traditional debug, to match with the understanding of the program or the problems.

4 EXPERIMENTATION

To validate our approach, we implemented the principles of our debugger semantics on the platform AnimUML [17] in order to try it on UML models. An example of the user interface during a debugging session is shown in Figure 5. On the left part, we can see the model view, where the user can choose among allowed actions (representing both step and select actions, depending if multiple choices are possible). On the right part, the history view shows the already explored configurations, with the possibility to jump back to any of them. The run to breakpoint function is hidden under the analysis tool menu, along with the possibility to create and select reduction functions.

AnimUML as a debugger also includes several useful functions which won't be expanded on further in this paper. As an example, it allows sharing debug sessions by exporting the model as url-links

or HTML files. This allows sending the debug session to another user, which could either analyze the trace, or continue the debug session.

The rest of this section describes two debug scenarios. The first scenario illustrates the use of different reduction policies on a slightly modified bakery mutual exclusion algorithm, an infinite state-space model. The second scenario, more performance oriented, shows the performance impact of different reduction policies.

4.1 Interests of the approach on infinite models

To show the interest of this approach we introduce the model of a mutual exclusion algorithm, based on Lamport's bakery algorithm [20] presented in the left part, the model section of Figure 5. Its base principle is based on a numbering machine giving each process entering the waiting section a new ticket with a higher value than the one previously distributed, and then ordering the processes by ascending ticket number. It presents a double interest: Firstly, modeling a non-deterministic behavior as the system can have multiple execution paths based on the scheduling of these two processes. Secondly, the presence of the incrementing tickets creates a possible execution path of ever increasing values and thus an infinite state space, making traditional model checking impossible.

To this model we also add another function: A counter *CounterCS* on one of the processes which increments each time it enters the critical section. As this counter is unbounded, it also makes the state space infinite. and will be used to show the possibilities of useless variables reductions.

As a first interesting element, the breakpoint syntax allows us a richer set of breakpoint definitions. We can define classic state-based breakpoints, equivalent to stopping on a given line of execution. In our example we could define it as processA is in state Waiting and processB is in the Critical Section. The extension of the syntax allows to extend breakpoint definition to more precise elements, such as values taken by a variable, or some message sent between objects. Like this, we could define a breakpoint on a certain value for one of the tickets, to reach it automatically without stepping manually for a long time.

Such a breakpoint placed on a given value of one of the tickets also demonstrates the interest of a non-deterministic debugging environment. With the presence of the tickets resetting functions when transitioning from the critical section to the initial state, the only way for the tickets value to continue increasing is when the two processes are perfectly coordinated, always going one after the other before the reset is effective. If we consider as a bug the value increasing too much (for example by going over the max value of the type chosen in the implementation language), a traditional debugger may have missed this specific sequence of events. By exploring all possible non-deterministic paths, the finder function will be able to find these special cases.

Reduction functions could be used to reach them faster. In this example, if we are only interested in the value of one of the tickets, we could define an abstraction which keeps one ticket as a concrete value, while ignoring or reducing the definition of the other one. Another reduction could be to outright ignore the counterCS variable, removing it from the configurations without changing the functional execution of the model.

Of course, the state configuration breakpoint that would be the most interesting to reach would be that both processes are in the critical section, but as this is a working mutual exclusion algorithm as well as a model with an infinite number of configurations, it is not possible to obtain a conclusion without some changes. This is the kind of situation where our reduction function can also be useful, by allowing us to gain more confidence, and in some cases a proof of a given property in the context of the explosion of the state space during exploration.

In addition to the methods already seen, we can define a reduction function on the model by using an exact abstraction with regards to the breakpoint. As an example of this situation, it is possible to define an abstraction function that reduces the state space enough to make it fully explorable, all the while keeping an equivalence with the concrete model. In our example, a reduction function computing the relation between ticketA and ticketB, as well as a boolean indicating if both tickets are at value 0, as described in the frame in the bottom right of 5.

The state space obtained by the exploration using this abstraction is shown the left part of Figure 5. This reduction function allows us to verify some properties on a model that is closer to a Petterson mutual exclusion algorithm [28], which has the advantage of having a finite number of configurations, while never modifying the original model. We can now try to reach a breakpoint describing both processes in a critical section, which effectively never happens, and the exploration can end.

4.2 Breakpoint reachability performances

In the previous section we used the reduction function to debug infinite models. Another usecase can be reducing the time or memory necessary to reach a given breakpoint. These types of breakpoints can already be reached using classical exploration algorithms, but they may either take too much resources, or simply be too slow to be comfortable to use in a debugging process. We can use a reduction function to reach them faster by skipping unnecessary parts of the exploration.

To evaluate the performances of the finder function and the possible gains offered by reductions, we propose a demonstration on a simple model displaying an explosion problem. This model, as shown in Figure 6, presents a simple non-deterministic environment in the form of a user which can send any value contained within a certain range to the system. This system is constrained to a smaller range of values to proceed with the rest of its operations. The environment is created with a int counter which can be incremented or decremented, with its value sent at any time to the system.

Our goal is to reach the "Breakpoint state", the configuration where the system can continue its execution with its higher values, without having to either wait for a long time, or to navigate manually through the problem. In this case, the state-space explosion problem comes from the different variables named value, stored as int ranging from 0 to 250, and of which multiple variations are present through the program. As all of these variables can change values independently, the number of combinations explodes making reachability analysis unnecessary complex.

Without further analyzing the problem, we can try to reach our breakpoint faster by reusing a hashing function. An example of

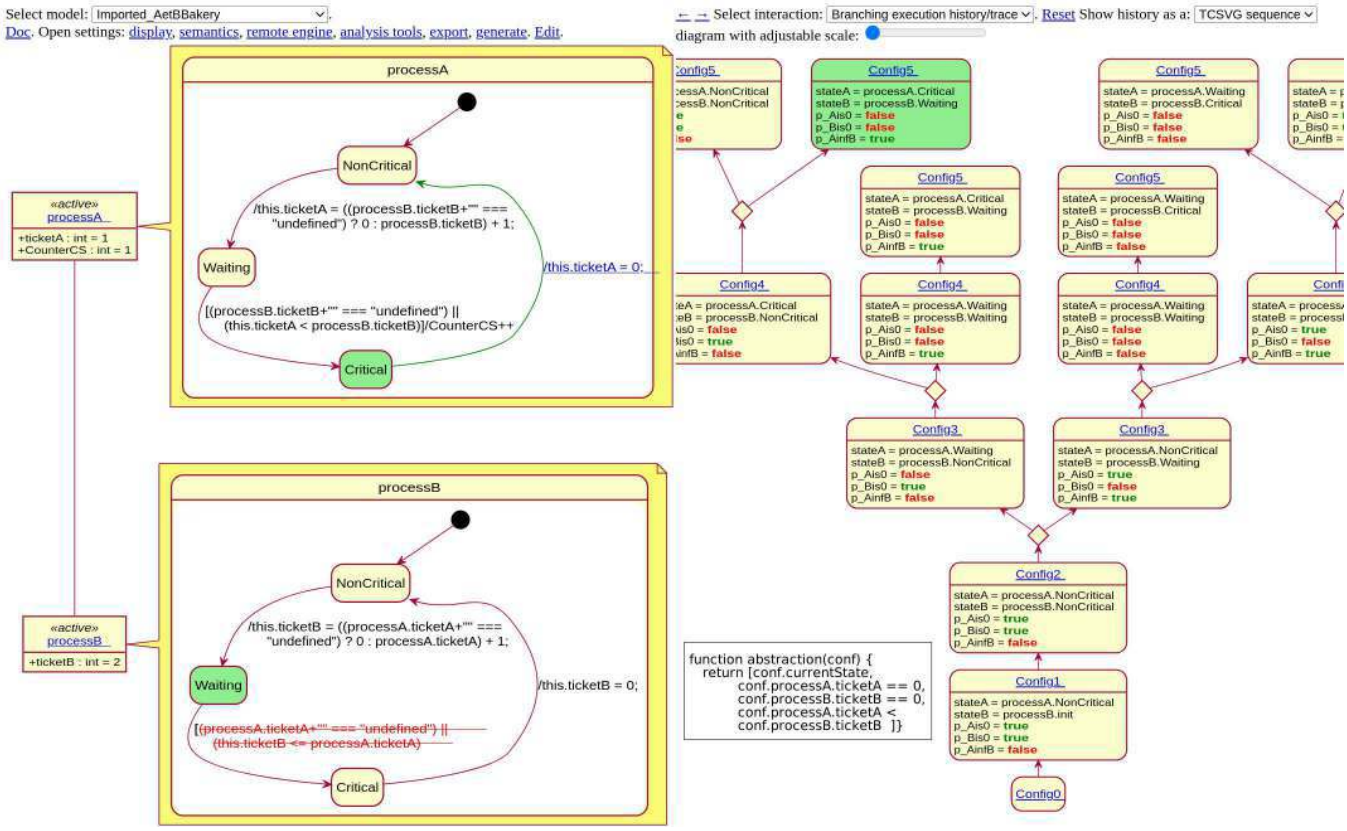

Figure 5: View of the AnimUML tool during a debugging session while using a reduction function.

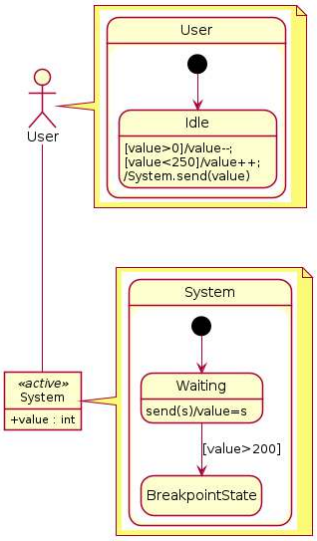

Figure 6: A non-deterministic model that challenges multi-verse debugging

the reduction obtained using this method is presented in Table 1. This table presents different metrics obtained by executing a run_to_breakpoint operation from the initial configuration, while using different reduction functions. For each run, we present the number of configurations explored as well as the time taken to reach it. Then, we display the time taken to explore one configuration, to have an estimation of the additional time used for computing the abstraction function. Finally, we can display the benefits of each reduction, in the form of the speedup in time it took to reach the breakpoint, by comparison with the run with no reduction. This table shows that, given a suitable reduction function, the additional time needed to execute the reduction function for each configuration still makes this approach worth it as it is compensated by the reduction in the number of configurations we need to explore.

If we want to go further in the reduction, we can start to act on the causes of the state space augmentation. An example of a problem caused by our modelisation is that for each configuration of the system, the environment will be explored with each possible value. The reduction function allows us to create classes of equivalent values, but we need a way to keep a trace between each configuration where the environment value is increasing to reach the higher values needed for our breakpoint. However we don't need to explore all environment values for each system one, and we can restrict the environment value by only storing the difference between these two, as shown in Listing 10 . As the environment value can only be incremented one by one, each configuration where the environment is going beyond will be linked to the superior or

Reduction function	Configurations	Time (ms)	Time per configuration (ms)	Speedup
No reduction	60914	851767	13,983	1
Sha-256, 16 bits	8931	168704	18,889	5.04
Sha-256, 15 bits	6121	119573	19,534	7,13
Abstraction 1	1213	37982	31,312	22,47
Abstraction 2	414	12777	30,862	66,66

Table 1: Results of breakpoint reachability using various reduction functions

```
1 function abstraction1(conf) {
2   return [conf.currentState,
3     conf.System.EventPool,
4     conf.User.value,
5     conf.User.value < conf.System.value,
6     conf.User.value > conf.System.value ]}
```

Listing 10: Reduction function based on limiting the maximum difference in the value between the user and the system

```
1 function abstraction2(conf) {
2   return [conf.currentState,
3           conf.User.value,
4           conf.System.EventPool.isEmpty(),
5           conf.System.value > 200 ]}
```

Listing 11: Reduction function based on classes of equivalent values

inferior configurations, thus stopping further exploration. With this reduction, we will not try every combination of successive environment value sent to the system, but as we are only trying to reach a certain value inside the system, this reduction will work fine, as we can see by the results of Table 1 under the line Abstraction 1.

Another more drastic reduction would involve ignoring all values except the specific one we are targeting. Of course, this only works in the specific case where a non-deterministic element of the model can send any possible values at any time, but in many cases, this corresponds to the definition of our environment modeling, built to test the limits of the system. Such a reduction function is presented in Listing 11. In this cases the value of the system is stored in the configuration as two boolean values:

- The first one is to indicate if a new message has been received and allows us to make a difference between configurations where a message is sent from the environment to the system, and those where the value is effectively read by the system. It also prevents the environment to keep sending useless messages, recreating the hypothesis of a reactive system.
- The second boolean value is simply to indicate if the received value is over or under a certain threshold, effectively reducing the variation of the system value to two possibilities.

These two combined reductions limits the possible states of the system to four before reaching the breakpoint state and by doing so limits the number of steps to reach it. The effects of this reduction can also be found in Table 1, under Abstraction 2.

5 LIMITS

We see several limits to our approach. Firstly, there are some limitations in our specific implementation related to action atomicity. The precise fault location can be hidden if a variable changes multiple times in a single action. Furthermore, there exist other techniques that improve the scalability of omniscient debugging, which could be applied to multiverse debugging. For instance, trace reductions techniques, which do not store every configurations in the history [25, 29], would further improve our approach. Furthermore, AnimUML has some limitations, which in turn limit our prototype. However, by adopting the SLI interface, we ensure that any language-side improvements are automatically supported.

Concerning the formalization itself, as we have seen, it is based on the SLI interface. If for whatever reason, the target language implementation is not compatible with this interface, our debugging semantics is no longer applicable. Some suppositions are also made about the SLI given for debugging. For example, we suppose every set in the STR is finite. As we iterate over them in the formalization we must come to an end to create the corresponding debug actions. Currently, the debug semantics, can operate on infinite state spaces, but not on infinite possibilities locally for a transition.

Then, another interesting possibility would be to directly use the lean formalization as the debugger, avoiding the complex and error-prone task of re-implementing everything for the target language.

Finally, the biggest issue with scalability is the choice of a reduction function, whose complexity may increase to provide significant effects on more complex models. While deterministic methods are easily transposed, targeted reductions may require more work to accounts for each reason for the space state explosion. This question will be treated further in our perspectives.

6 CONCLUSION & PERSPECTIVES

This article presented an extension of multiverse debugging using reduction techniques to facilitate the breakpoint search process. We completed a formalization of a debug semantics for non-deterministic specifications languages. We finally presented a UML debugging environment, that allows debugging models that would normally be too complex for classic multiverse debugging tools.

One of the challenges that needs to be addressed is helping the user choose a suitable reduction function for the model and its diagnosis needs. This problem can be further decomposed in three questions: *1) What are the typical causes of state-space explosion on models? 2) Are there domain-specific reductions that can be used for specific classes of models, or model patterns? 3) Can these reductions be applied automatically and adapted to the context of each model?*

REFERENCES

[1] Martín Abadi and Leslie Lamport. 1991. The existence of refinement mappings. *Theoretical Computer Science* 82, 2 (1991), 253–284. https://doi.org/10.1016/0304-3975(91)90224-P

[2] Kelly Androutsopoulos, David Clark, Mark Harman, Jens Krinke, and Laurence Tratt. 2013. State-Based Model Slicing: A Survey. *ACM Comput. Surv.* 45, 4, Article 53 (aug 2013), 36 pages. https://doi.org/10.1145/2501654.2501667

[3] Mojtaba Bagherzadeh, Nicolas Hili, David Seekatz, and Juergen Dingel. 2018. MDebugger: a model-level debugger for UML-RT. https://doi.org/10.1145/3183440.3183473

[4] Mojtaba Bagherzadeh, Karim Jahed, Benoit Combemale, and Juergen Dingel. 2019. Live-UMLRT: A Tool for Live Modeling of UML-RT Models. In *MODELS 2019 - ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems*. IEEE, Munich, Germany, 743–747. https://doi.org/10.1109/MODELS-C.2019.00115

[5] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. *ACM Comput. Surv.* 51, 3, Article 50 (may 2018), 39 pages. https://doi.org/10.1145/3182657

[6] Valentin Besnard. 2020. *EMI: An approach to unify analysis and embedded execution with a controllable model interpreter*. Ph. D. Dissertation. ENSTA Bretagne.

[7] Valentin Besnard, Ciprian Teodorov, Frédéric Jouault, Matthias Brun, and Philippe Dhaussy. 2021. Unified verification and monitoring of executable UML specifications. *Software and Systems Modeling* 20, 6 (01 12 2021), 1825–1855. https://doi.org/10.1007/s10270-021-00923-9

[8] Bob Boothe. 2000. Efficient algorithms for bidirectional debugging. In *Proceedings of the ACM SIGPLAN 2000 conference on Programming language design and implementation*. 299–310.

[9] Erwan Bousse, Dorian Leroy, Benoit Combemale, Manuel Wimmer, and Benoit Baudry. 2018. Omniscient debugging for executable DSLs. *Journal of Systems and Software* 137 (2018), 261–288. https://doi.org/10.1016/j.jss.2017.11.025

[10] Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem. 2018. *Handbook of Model Checking* (1st ed.). Springer Publishing Company, Incorporated.

[11] Michael A. Colón and Tomás E. Uribe. 1998. Generating finite-state abstractions of reactive systems using decision procedures. In *Computer Aided Verification*, Alan J. Hu and Moshe Y. Vardi (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 293–304.

[12] David Darais, Nicholas Labich, Phúc C. Nguyen, and David Van Horn. 2017. Abstracting Definitional Interpreters (Functional Pearl). *Proc. ACM Program. Lang.* 1, ICFP, Article 12 (aug 2017), 25 pages. https://doi.org/10.1145/3110256

[13] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. The Lean Theorem Prover (System Description). In *Automated Deduction - CADE-25*, Amy P. Felty and Aart Middeldorp (Eds.). Springer International Publishing, Cham, 378–388.

[14] A. Georges, M. Christiaens, M. Ronsse, and K. De Bosschere. 2004. JaRec: A Portable Record/Replay Environment for Multi-Threaded Java Applications. *Softw. Pract. Exper.* 34, 6 (may 2004), 523–547. https://doi.org/10.1002/spe.579

[15] Susanne Graf and Hassen Saidi. 1997. Construction of abstract state graphs with PVS. In *Computer Aided Verification*, Orna Grumberg (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 72–83.

[16] Gerard J. Holzmann. 1996. *An analysis of bitstate hashing*. Springer US, Boston, MA, 301–314. https://doi.org/10.1007/978-0-387-34892-6_19

[17] Frédéric Jouault, Valentin Besnard, Théo Le Calvar, Ciprian Teodorov, Matthias Brun, and Jerome Delatour. 2020. Designing, Animating, and Verifying Partial UML Models. In *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems* (Virtual Event, Canada) *(MODELS '20)*. Association for Computing Machinery, New York, NY, USA, 211–217. https://doi.org/10.1145/3365438.3410967

[18] Naoki Kobayashi, Ryosuke Sato, and Hiroshi Unno. 2011. Predicate Abstraction and CEGAR for Higher-Order Model Checking. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) *(PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 222–233. https://doi.org/10.1145/1993498.1993525

[19] Naoki Kobayashi, Ryosuke Sato, and Hiroshi Unno. 2011. Predicate Abstraction and CEGAR for Higher-Order Model Checking. *SIGPLAN Not.* 46, 6 (jun 2011), 222–233. https://doi.org/10.1145/1993316.1993525

[20] Leslie Lamport. 1974. A New Solution of Dijkstra's Concurrent Programming Problem. *Commun. ACM* 17, 8 (aug 1974), 453–455. https://doi.org/10.1145/361082.361093

[21] Yoann Laurent, Reda Bendraou, and Marie-Pierre Gervais. 2013. Executing and debugging UML models: an fUML extension. In *SAC'13 - The 28th Annual ACM Symposium on Applied Computing*. ACM, Coimbra, Portugal, 1095–1102. https://doi.org/10.1145/2480362.2480569

[22] Bil Lewis. 2003. Debugging Backwards in Time. *Computing Research Repository* cs.SE/0310016 (10 2003).

[23] Carmen Torres Lopez, Robbert Gurdeep Singh, Stefan Marr, Elisa Gonzalez Boix, and Christophe Scholliers. 2019. Multiverse Debugging: Non-deterministic Debugging for Non-deterministic Programs. In *33rd European Conference on Object-Oriented Programming*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik. https://kar.kent.ac.uk/74328/

[24] Simon Van Mierlo, Yentl Van Tendeloo, and Hans Vangheluwe. 2017. Debugging Parallel DEVS. *SIMULATION* 93 (2017), 285 – 306.

[25] Oleg Y. Nickolayev, Philip C. Roth, and Daniel A. Reed. 1997. Real-Time Statistical Clustering for Event Trace Reduction. *The International Journal of Supercomputer Applications and High Performance Computing* 11, 2 (1997), 144–159. https://doi.org/10.1177/109434209701100207

[26] OMG. 2017. Unified Modeling Language. https://www.omg.org/spec/UML/2.5.1/PDF

[27] Corina S. Păsăreanu, Radek Pelánek, and Willem Visser. 2005. Concrete Model Checking with Abstract Matching and Refinement. In *Computer Aided Verification*, Kousha Etessami and Sriram K. Rajamani (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 52–66.

[28] Gary L. Peterson. 1981. Myths About the Mutual Exclusion Problem. *Inf. Process. Lett.* 12 (1981), 115–116.

[29] Guillaume Pothier, Éric Tanter, and José Piquer. 2007. Scalable omniscient debugging. *Sigplan Notices - SIGPLAN* 42, 535–552. https://doi.org/10.1145/1297105.1297067

[30] Tillmann Rendel, Klaus Ostermann, and Christian Hofer. 2009. Typed Self-Representation. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Dublin, Ireland) *(PLDI '09)*. Association for Computing Machinery, New York, NY, USA, 293–303. https://doi.org/10.1145/1542476.1542509

[31] Grigore Roșu and Traian Florin Șerbănuță. 2010. An overview of the K semantic framework. *The Journal of Logic and Algebraic Programming* 79, 6 (2010), 397–434. https://doi.org/10.1016/j.jlap.2010.03.012 Membrane computing and programming.

[32] M. Saxena. 2018. *A Language Independent Debugger Semantics Based Debugging in K*. Master's thesis. University of Illinois at Urbana-Champaign. https://www.ideals.illinois.edu/handle/2142/101590

[33] Ulrich Stern and David L. Dill. 1995. Improved probabilistic verification by hash compaction. In *Correct Hardware Design and Verification Methods*, Paolo E. Camurati and Hans Eveking (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 206–224.

[34] William Visser, SeungJoon Park, and John Penix. 2000. Using Predicate Abstraction to Reduce Object-Oriented Programs for Model Checking. In *Proceedings of the Third Workshop on Formal Methods in Software Practice* (Portland, Oregon, USA) *(FMSP '00)*. Association for Computing Machinery, New York, NY, USA, 3–182. https://doi.org/10.1145/349360.351125

[35] P. Wadler and S. Blott. 1989. How to Make Ad-Hoc Polymorphism Less Ad Hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Austin, Texas, USA) *(POPL '89)*. Association for Computing Machinery, New York, NY, USA, 60–76. https://doi.org/10.1145/75277.75283